

Classification

*Lecturer: Jiaming Liang**September 17, 2026*

1 Classification and Logistic Regression

In a classification problem, the output takes values in a finite set of classes, $y \in \{1, \dots, M\}$. A probabilistic classifier models the conditional class probabilities

$$g_m(\mathbf{x}) \approx p(y = m \mid \mathbf{x}), \quad m = 1, \dots, M,$$

where

$$g_m(\mathbf{x}) \geq 0, \quad \sum_{m=1}^M g_m(\mathbf{x}) = 1.$$

We first consider binary classification with $y \in \{-1, 1\}$.

Binary logistic regression. A linear score $z = \boldsymbol{\theta}^\top \mathbf{x}$ can take any real value, so it cannot directly represent a probability. We therefore transform it using the logistic, or sigmoid, function

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Logistic regression models

$$p(y = 1 \mid \mathbf{x}; \boldsymbol{\theta}) = g(\mathbf{x}; \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x}),$$

and

$$p(y = -1 \mid \mathbf{x}; \boldsymbol{\theta}) = 1 - g(\mathbf{x}; \boldsymbol{\theta}) = 1 - \sigma(\boldsymbol{\theta}^\top \mathbf{x}) = \sigma(-\boldsymbol{\theta}^\top \mathbf{x}).$$

Using $y_i \in \{-1, 1\}$, the probabilities of the two classes can be written compactly as

$$p(y_i \mid \mathbf{x}_i; \boldsymbol{\theta}) = \sigma\left(y_i \boldsymbol{\theta}^\top \mathbf{x}_i\right).$$

Maximum likelihood and cross-entropy. Given training data $\mathcal{T} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, we estimate the parameters by maximum likelihood:

$$\hat{\boldsymbol{\theta}} = \arg \max_{\boldsymbol{\theta}} \prod_{i=1}^n p(y_i \mid \mathbf{x}_i; \boldsymbol{\theta}).$$

Equivalently,

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} -\frac{1}{n} \sum_{i=1}^n \log p(y_i \mid \mathbf{x}_i; \boldsymbol{\theta}).$$

The negative log-likelihood can be interpreted as a *cross-entropy loss*. If $\mathbf{q}$ denotes the true class distribution and $\mathbf{p}$ the predicted distribution, then

$$H(\mathbf{q}, \mathbf{p}) = - \sum_{m=1}^M q_m \log p_m.$$

For a supervised example whose true class is y , the target distribution $\mathbf{q}$ is represented using *one-hot encoding*. This means

$$q_m = \begin{cases} 1, & m = y, \\ 0, & m \neq y. \end{cases}$$

In other words, all probability mass is placed on the true class. Therefore,

$$H(\mathbf{q}, \mathbf{p}) = - \sum_{m=1}^M q_m \log p_m = - \log p_y.$$

Thus cross-entropy is the negative logarithm of the probability assigned to the correct class.

For binary logistic regression,

$$p(y_i | \mathbf{x}_i; \boldsymbol{\theta}) = \frac{1}{1 + e^{-y_i \boldsymbol{\theta}^\top \mathbf{x}_i}},$$

and therefore

$$\ell_i(\boldsymbol{\theta}) = - \log p(y_i | \mathbf{x}_i; \boldsymbol{\theta}) = \log \left(1 + e^{-y_i \boldsymbol{\theta}^\top \mathbf{x}_i} \right).$$

This is the *logistic loss*. The training problem is

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \quad J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \log \left(1 + e^{-y_i \boldsymbol{\theta}^\top \mathbf{x}_i} \right).$$

Why not minimize misclassification loss directly? A natural classification loss is the 0-1 loss

$$\ell_{0-1}(y, \hat{y}) = \mathbf{1}\{\hat{y} \neq y\}.$$

Since the predicted label depends on the model parameters, for a linear classifier

$$\hat{y}_{\boldsymbol{\theta}}(\mathbf{x}) = \text{sign} \left(\boldsymbol{\theta}^\top \mathbf{x} \right),$$

the loss can equivalently be written as

$$\ell_{0-1}(\boldsymbol{\theta}; \mathbf{x}, y) = \mathbf{1} \left\{ y \boldsymbol{\theta}^\top \mathbf{x} \leq 0 \right\}.$$

Although the misclassification rate is often the final quantity of interest, the 0-1 loss is difficult to optimize directly. As a function of $\boldsymbol{\theta}$, it is piecewise constant: small changes in $\boldsymbol{\theta}$ do not change

the loss unless a sample crosses the decision boundary. Consequently, its gradient is zero almost everywhere and is not defined at the points where the predicted class changes.

Moreover, the 0-1 loss only distinguishes between correct and incorrect predictions. It assigns the same loss to a correctly classified point very close to the decision boundary and one classified with high confidence. Cross-entropy provides a smooth alternative that also takes the predicted probabilities into account.

Margin interpretation. A binary classifier can be written as

$$\hat{y}(\mathbf{x}) = \text{sign}(f(\mathbf{x})).$$

For an observation $(\mathbf{x}, y)$, the quantity $yf(\mathbf{x})$ is called its *margin*. A positive margin corresponds to a correct classification, while a negative margin corresponds to a misclassification.

For logistic regression, $f(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x}$, and

$$\ell_i(\boldsymbol{\theta}) = \log \left(1 + e^{-y_i f(\mathbf{x}_i)} \right).$$

Thus logistic loss is a smooth decreasing function of the classification margin: larger positive margins receive smaller loss.

Gradient, Hessian, and convexity. Let $z_i = y_i \boldsymbol{\theta}^\top \mathbf{x}_i$. Since $\ell_i(\boldsymbol{\theta}) = \log(1 + e^{-z_i})$, we have

$$\nabla \ell_i(\boldsymbol{\theta}) = -y_i \sigma(-z_i) \mathbf{x}_i.$$

Therefore,

$$\nabla J(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^n y_i \sigma(-z_i) \mathbf{x}_i.$$

Differentiating once more,

$$\nabla^2 \ell_i(\boldsymbol{\theta}) = \sigma(z_i) \sigma(-z_i) \mathbf{x}_i \mathbf{x}_i^\top,$$

and hence

$$\nabla^2 J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \sigma(z_i) \sigma(-z_i) \mathbf{x}_i \mathbf{x}_i^\top.$$

For any vector $\mathbf{v}$,

$$\mathbf{v}^\top \nabla^2 J(\boldsymbol{\theta}) \mathbf{v} = \frac{1}{n} \sum_{i=1}^n \sigma(z_i) \sigma(-z_i) (\mathbf{x}_i^\top \mathbf{v})^2 \geq 0.$$

Therefore,

$$\nabla^2 J(\boldsymbol{\theta}) \succeq 0,$$

so logistic regression is a convex optimization problem. Unlike least squares, however, the optimality condition $\nabla J(\boldsymbol{\theta}) = 0$ is nonlinear in $\boldsymbol{\theta}$ and does not yield a closed-form solution. Logistic regression is therefore typically trained using iterative optimization methods such as gradient descent or Newton's method.

2 Predictions and Decision Boundaries

Logistic regression produces class probabilities rather than class labels. To obtain a hard prediction, we choose a threshold $r \in (0, 1)$ and define

$$\hat{y}(\mathbf{x}) = \begin{cases} 1, & g(\mathbf{x}; \boldsymbol{\theta}) > r, \\ -1, & g(\mathbf{x}; \boldsymbol{\theta}) \leq r. \end{cases}$$

The standard choice is $r = \frac{1}{2}$. To understand why, suppose for the moment that

$$g(\mathbf{x}) = p(y = 1 \mid \mathbf{x})$$

is the true conditional probability.

For a fixed input $\mathbf{x}$, there are only two possible predictions. If we predict 1, we make an error when the true label is -1 , so

$$\mathbb{P}(\text{error} \mid X = \mathbf{x}, \hat{y} = 1) = 1 - g(\mathbf{x}).$$

If we predict -1 , we make an error when the true label is 1, so

$$\mathbb{P}(\text{error} \mid X = \mathbf{x}, \hat{y} = -1) = g(\mathbf{x}).$$

Therefore, we should predict the class with the smaller probability of error:

$$\hat{y}(\mathbf{x}) = \begin{cases} 1, & g(\mathbf{x}) > \frac{1}{2}, \\ -1, & g(\mathbf{x}) < \frac{1}{2}. \end{cases}$$

Thus the threshold $r = 1/2$ simply chooses the more probable class.

For logistic regression,

$$g(\mathbf{x}; \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^\top \mathbf{x}}}.$$

The decision boundary for the threshold $r = 1/2$ consists of all points for which $g(\mathbf{x}; \boldsymbol{\theta}) = \frac{1}{2}$. Since $\sigma(z) = \frac{1}{2}$ is equivalent to $z = 0$, the decision boundary is

$$\boldsymbol{\theta}^\top \mathbf{x} = 0.$$

Therefore logistic regression is a *linear classifier*: its decision boundary is a hyperplane. Moreover,

$$\hat{y}(\mathbf{x}) = \text{sign}(\boldsymbol{\theta}^\top \mathbf{x}).$$

For a general threshold $r \in (0, 1)$, the decision boundary satisfies $g(\mathbf{x}; \boldsymbol{\theta}) = r$, which gives

$$\boldsymbol{\theta}^\top \mathbf{x} = \log \frac{r}{1-r}.$$

Thus changing the threshold shifts the decision boundary while preserving its linear form.

Different costs for different errors. The threshold $r = 1/2$ treats the two types of classification errors equally. In some applications, however, false positives and false negatives may have very different consequences.

Let C_{FP} denote the cost of a *false positive*, i.e., predicting 1 when the true label is -1 , and let C_{FN} denote the cost of a *false negative*, i.e., predicting -1 when the true label is 1.

For a fixed input $\mathbf{x}$, predicting 1 has expected cost $C_{\text{FP}}[1 - g(\mathbf{x})]$, while predicting -1 has expected cost $C_{\text{FN}}g(\mathbf{x})$. Therefore, we should predict 1 when

$$C_{\text{FP}}[1 - g(\mathbf{x})] < C_{\text{FN}}g(\mathbf{x}),$$

or equivalently,

$$g(\mathbf{x}) > \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}}.$$

Hence the optimal threshold is

$$r^* = \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}}.$$

When the two errors have equal cost $C_{\text{FP}} = C_{\text{FN}}$, we recover $r^* = \frac{1}{2}$.

For example, consider a medical screening problem where class 1 means that a patient has a disease. A false negative corresponds to a missed diagnosis, while a false positive corresponds to an unnecessary follow-up test. If a missed diagnosis is much more costly than a false alarm, then $C_{\text{FN}} > C_{\text{FP}}$ which gives $r^* < \frac{1}{2}$. The classifier therefore uses a lower threshold for predicting the positive class, reducing the chance of missing a patient who has the disease.

3 Multiclass Classification

We now consider $y \in \{1, \dots, M\}$, $M > 2$. For each class m , define a linear score, or *logit*, $z_m = \boldsymbol{\theta}_m^\top \mathbf{x}$. These scores can take arbitrary real values and therefore cannot directly be interpreted as probabilities. The *softmax function* converts them into a probability distribution:

$$g_m(\mathbf{x}; \boldsymbol{\theta}) = \frac{e^{\boldsymbol{\theta}_m^\top \mathbf{x}}}{\sum_{j=1}^M e^{\boldsymbol{\theta}_j^\top \mathbf{x}}}, \quad m = 1, \dots, M.$$

The probabilities are positive and sum to one.

The predicted class is

$$\hat{y}(\mathbf{x}) = \arg \max_m g_m(\mathbf{x}; \boldsymbol{\theta}).$$

Since all classes share the same denominator and the exponential function is strictly increasing,

$$\hat{y}(\mathbf{x}) = \arg \max_m \boldsymbol{\theta}_m^\top \mathbf{x}.$$

As in the binary case, the model is trained by maximum likelihood. For a training example $(\mathbf{x}_i, y_i)$, the probability assigned to the true class is

$$g_{y_i}(\mathbf{x}_i; \boldsymbol{\theta}),$$

so the cross-entropy loss is

$$L_i = -\log g_{y_i}(\mathbf{x}_i; \boldsymbol{\theta}).$$

Equivalently, if $\mathbf{q}_i$ is the one-hot representation of the true class,

$$L_i = -\sum_{m=1}^M q_{im} \log g_m(\mathbf{x}_i; \boldsymbol{\theta}).$$

Substituting the softmax probabilities gives

$$L_i = -\log \frac{e^{\boldsymbol{\theta}_{y_i}^\top \mathbf{x}_i}}{\sum_{j=1}^M e^{\boldsymbol{\theta}_j^\top \mathbf{x}_i}} = -\boldsymbol{\theta}_{y_i}^\top \mathbf{x}_i + \log \left(\sum_{j=1}^M e^{\boldsymbol{\theta}_j^\top \mathbf{x}_i} \right).$$

Therefore the multiclass logistic regression objective is

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \left[-\boldsymbol{\theta}_{y_i}^\top \mathbf{x}_i + \log \left(\sum_{j=1}^M e^{\boldsymbol{\theta}_j^\top \mathbf{x}_i} \right) \right].$$

The boundary between two classes m and k is determined by $g_m(\mathbf{x}) = g_k(\mathbf{x})$, which is equivalent to $(\boldsymbol{\theta}_m - \boldsymbol{\theta}_k)^\top \mathbf{x} = 0$. Thus multiclass logistic regression also has linear decision boundaries.

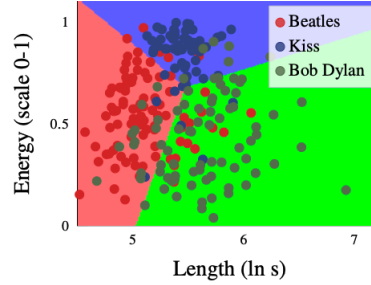


Figure 1: Logistic regression for music classification.

Note: Binary cross-entropy is the $M = 2$ special case of multiclass cross-entropy. Let

$$\mathbf{g}(\mathbf{x}) = \begin{bmatrix} g(\mathbf{x}) \\ 1 - g(\mathbf{x}) \end{bmatrix}.$$

Then

$$-\log g_{y_i}(\mathbf{x}_i) = \begin{cases} -\log g(\mathbf{x}_i), & y_i = 1, \\ -\log(1 - g(\mathbf{x}_i)), & y_i = -1, \end{cases}$$

which is exactly the binary cross-entropy loss.